

0.11. Tout l'alphabet, avec textS

textS passe à la ligne tout seul : plus de calcul à écrire.

1. Ce qu'il faut comprendre

`textS` écrit comme `texte`, mais **pass**e à la ligne quand la ligne est pleine : après la colonne 19, la suite reprend en colonne 0 de la ligne d'en dessous.

Le calcul du 0.9 (`i % 20`, `i / 20`) est fait à **notre place**. `texte()`, lui, refuserait ce mot trop long.

Après la ligne 17, `textS` repart **en haut**, à la ligne 0.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <textS>` → [0.10.3. La fonction textS\(\) — un texte qui passe à la ligne](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **639 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage – la mise en route de la console
main()	29 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
CopierTuiles	31 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	416 o	← CopierTuiles – 26 lettres sur 43
"ABCDEFGHJKLMNO..."	20 o	← texte() – un texte gravé
"UVWXYZ"	6 o	← texte() – un texte gravé

Lettres gravées : ABCDEFGHIJKLMNOPQRSTUVWXYZ

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte() × 2	22 o sur place, dans main()	+ EcrireTexte (10 o)
image() × 1	3 o sur place, dans main()	+ AttendreImage (28 o)
textS() × 1	0 o sur place, dans main()	

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <textS> // écrit un texte qui passe à la ligne tout seul
```

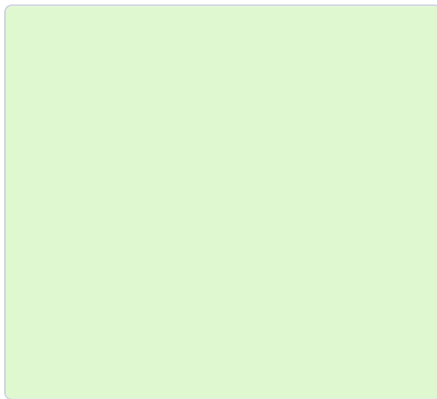
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                     // Le programme commence ici.
    textS(0, 0, "ABCDEFGHIJKLMNOPQRSTUVWXYZ"); // passe à la ligne après T
                                                // textS(colonne, ligne, "TEXTE") : comme
                                                // texte(), mais quand le texte arrive au
                                                // bord droit (colonne 19), il continue
                                                // tout seul au début de la ligne suivante.
                                                // A à T sur la ligne 0, U à Z sur la ligne 1.

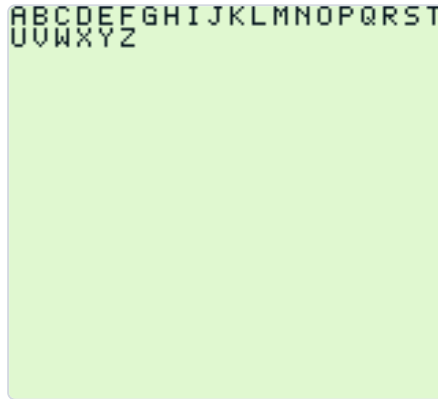
    while (true) {                               // La boucle de jeu : pour toujours...
        image();                                 // ... attendre l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur la ligne 0, U à Z en dessous, comme au 0.9.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la ligne 0 : A à T

✓ la ligne 1 : U à Z

Ensuite : **0.11.1. Tout l'alphabet, avec textS — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++